DevOps Explanation Request

Alright vicky 😈
Today's episode: **Kubernetes TLS Certificates & CSR** — the part everyone *uses*, nobody *understands*, and interviews *love*.

I'll keep it **easy English**, DevOps-clean, and follow your rule:

👉 **way → problem → how it solves → steps**

☠️ Way (What would happen WITHOUT TLS)

- User talks to kube-api-server in **plain text**
- Control plane talks to worker nodes **without encryption**
- Anyone on the network can:
  - Read data
  - Modify requests
  - Impersonate components

Basically:
Production cluster running like public Wi-Fi at a railway station.

💣 Problem

1. Kubernetes is **distributed**
2. Many components talk to each other:
   - user ↔ API server
   - API server ↔ kubelet
   - API server ↔ etcd
3. Without trust + encryption:
   - Identity can't be verified
   - Data can be sniffed or changed

🛠️ How Kubernetes solves this

Using **TLS + Certificates**

Just like HTTPS, but **every component is paranoid** (as it should be).

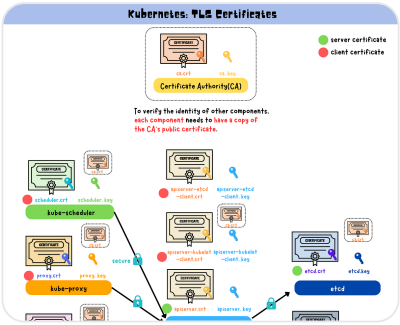
Step 1 – Overview (High-level understanding)

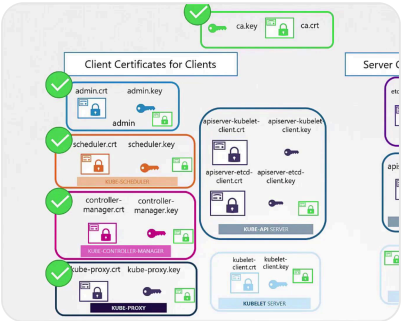
Kubernetes uses **TLS everywhere** for:

- Authentication (who are you?)
- Encryption (hide data)
- Trust (should I talk to you?)

There are **3 types of certificates** 👇

Certificate Type	Used by
Client Certificate	Users, kubelet, controllers
Server Certificate	kube-api-server, kubelet
CA Certificate	Signs & verifies all certs





Step 2 – Core Concepts

🔹 Client–Server model (VERY IMPORTANT)

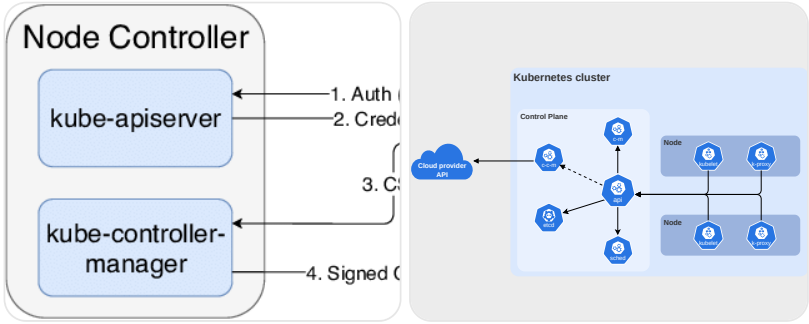
Case 1: User → Control Plane

- User = client
- API Server = server
- Needs TLS ✔️

Case 2: Control Plane → Worker Node

- **API Server = client**
- **Kubelet = server**
- Needs TLS ✔️

So roles **change based on communication**, but TLS is **always ON**.



🔐 Certificate Authority (CA)

CA is the **boss of trust**.

What CA does:

1. Receives CSR (Certificate Signing Request)
2. Verifies identity
3. Signs certificate
4. Everyone trusts CA → cert becomes valid

If cert is **not signed by trusted CA** ×
👉 Kubernetes says: *"Who are you? Get lost."*

Step 3 – Hands-on Example (CSR Flow)

We'll create a **user certificate** (`adam`) step by step.

1️⃣ Generate private key

```bash
openssl genrsa -out adam.key 2048
```

👉 This is **adam's identity**

👉 NEVER share this key (or your cluster cries)

2️⃣ Generate CSR (Certificate Signing Request)

```bash
openssl req -new -key adam.key -out adam.csr -subj "/CN=adam"
```

What this means:

- `CN=adam` → username inside Kubernetes
- CSR = "Please sign my identity"

3️⃣ Create CSR object in Kubernetes

(Kubernetes CSR object contains **base64 CSR**)

```yaml
apiVersion: certificates.k8s.io/v1
kind: CertificateSigningRequest
metadata:
  name: adam-csr
spec:
  request: <base64-of-adam.csr>
  signerName: kubernetes.io/kube-apiserver-client
  usages:
  - client auth
```

4️⃣ Approve or deny CSR

Approve ✅

```bash
kubectl certificate approve adam-csr
```

Deny ×

```bash
kubectl certificate deny adam-csr
```

Once approved:

👉 Kubernetes signs it using **CA**

👉 User gets a valid client certificate

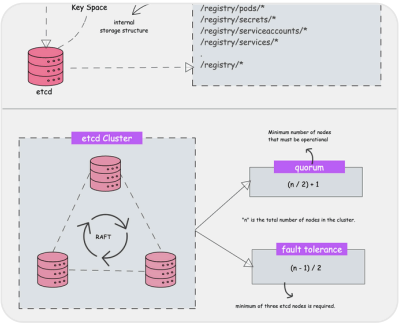
Step 4 – How certs are actually loaded

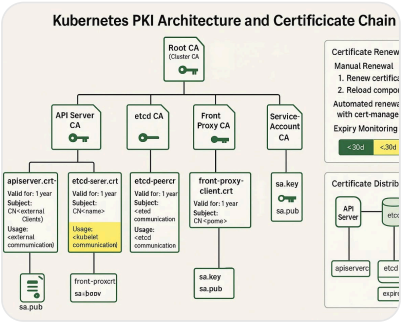
Control-plane components run as **static pods**

Certificates are mounted as **volumes**.

Example (etcd):

- `/etc/kubernetes/pki/`
- certs + keys mounted inside pod
- kubelet loads them at startup





Step 5 – Where certs are used (Control Plane)

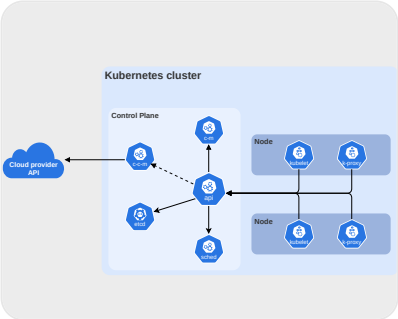
Every major component has:

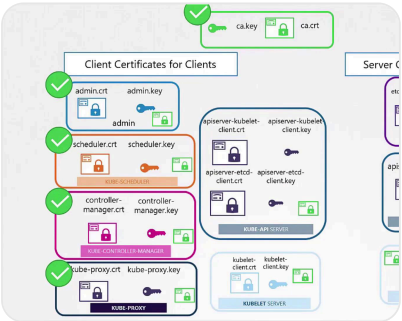
- its **own cert**
- its **own private key**

Components using TLS:

- kube-apiserver
- kube-scheduler
- controller-manager
- etcd

- kubelet
- kube-proxy
- users (kubectl)





No cert = no communication 🚫

Step 6 – Common Mistakes (Interview Gold 🥇)

× Thinking TLS is only for users

× Confusing client vs server roles

× Forgetting CN maps to Kubernetes username

× Assuming cert approval is automatic

× Losing private keys (game over)

Step 7 – Advanced / Real-World Notes

🔥 kubeadm automatically:

- creates CA
- generates all control-plane certs
- rotates some certs

🔥 In production:

- CSR approval is often automated
- RBAC is mapped to cert CN
- External CA / Vault may be used

🔥 Certificate rotation matters:

- expired cert = cluster outage 💀

🧠 Final Mental Model (lock this in)

```ini
TLS = Encryption + Identity + Trust
CA  = Trust Anchor
CSR = “Please sign my identity”
```

Or even simpler:

> **Kubernetes talks only to identities it trusts, over encrypted channels.**

If you want next:

- TLS vs RBAC (auth vs authz)
- kubeconfig + certs deep dive
- Cert rotation & expiry
- Debugging “x509 certificate signed by unknown authority”

Say the word 😏🔥